

Chapter 5 — Composition

Chapter 5 — Composition

A 200-line component is a smell. So is splitting too early. This chapter teaches you the senior judgement of when to extract a component, when to leave one alone, and how to build deeply nested feature views without losing the plot.

What you'll learn

- The **copy-paste rule** — when a chunk of JSX deserves its own component.
- **Tiny components for readability** (`<PathArrow>`, `<PathStep>`) — when zero-prop components are *good* code.
- **Prop drilling** — how deep is too deep, and how to know.
- **Context** — when to reach for it, when not to.
- How to build a **deeply nested feature** — the Players tab — by composition, not by giant monolithic files.
- The two-way **back/forward navigation** pattern between sibling views.

Lessons in this chapter

1. [01-when-to-split-components.md](#) — The judgement call: when a chunk of JSX deserves a name. The copy-paste rule. The “what vs how” rule. Tiny components and the case for them.
2. [02-prop-drilling-vs-context.md](#) — When to keep drilling, when to introduce Context, why most React advice on this topic is wrong, and what senior engineers actually do.
3. [03-building-player-detail.md](#) — A real case study: the 270-line `<PlayerDetail>` component, the four sub-components it composes, and the design decisions behind

each split.

Files you'll create or update

```
components/
├── matches/                                ← MatchesTab fully built out
│   ├── MatchesList.tsx
│   ├── PlayerLine.tsx
│   ├── RoundButton.tsx
│   └── UpcomingMatchList.tsx
├── predictions/                            ← PredictionsTab fully built out
│   ├── PredictionMatrix.tsx
│   ├── TeamCardView.tsx
│   ├── PicksList.tsx
│   ├── RoundStat.tsx
│   └── Legend.tsx
├── players/                               ← PlayersTab + PlayerDetail
│   ├── PlayerCard.tsx
│   ├── PlayerDetail.tsx                  ← the big one
│   ├── MatchPickAnalysis.tsx
│   ├── PathStep.tsx
│   ├── PathArrow.tsx
│   ├── StatTile.tsx
│   └── TeamChip.tsx
└── tabs/
    ├── MatchesTab.tsx                    ← UPDATED: real implementation
    ├── PredictionsTab.tsx                ← UPDATED
    └── PlayersTab.tsx                    ← UPDATED
```

That's a lot of files. Most are 30–60 lines each. The point of the chapter is **making lots of small files instead of a few big ones** — and learning to defend that choice.

New React/Next.js concepts introduced

- **Component extraction** — moving JSX into a sibling component file.
- **Composition** — building complex views by nesting simple components.
- **children prop** — passing JSX as a prop.

- **Optional callbacks for navigation** (onSelectPlayer, onBack).
- **Local state in tab components** (useState for the round filter, the player drill-down).
- **Conditional rendering of detail views** (if selectedPlayer return <PlayerDetail />).
- **Reusable pure-presentation components** with no business logic.

How long it should take

- 30–45 min reading per lesson
- 2–3 hours typing the code (lots of files, but each is short)
- ~5 hours total

What you'll see at the end

After this chapter:

- **Matches tab** with a round filter and cards for every match.
- **Predictions tab** with a comparison matrix and a single-team card view.
- **Players tab** with a 32-player grid and click-through to a deep detail view.

Three of the five tabs fully working. Only Analytics (Chapter 6) and final polish remain.

Before you start

You should have:

- Chapter 4 complete: orchestrator + state + tab nav.
- All five tabs at least stubbed.
- The Standings tab fully working.

Open [01-when-to-split-components.md](#).